Cuatro LED en la breadboard: la luz que corre

Lección 9 — 1MAK, SST-INF-01c — Circuito ordenado en la breadboard, colores de los cables, setup y loop escritos desde cero, secuencia de LED, leer los errores

03/11/2026

1 Desde dónde empezamos

La última vez conectamos a Arduino **un LED** y después **cuatro LED** “en el aire”, y conocimos la **breadboard**. Hoy volvemos a hacer el circuito con los cuatro LED **en la breadboard** y escribimos el programa **desde cero**, para que la luz corra de un LED al otro.

Nota

Para entrar en Tinkercad, el navegador debe estar conectado a su cuenta de la escuela. Arriba a la derecha, **cerca de la X de cierre**, puede aparecer el botón **Verifica la tua identità** (verifica tu identidad): hay que hacer clic para iniciar sesión. No basta con entrar desde el icono dentro del sitio. Tinkercad se escribe **con K**.

2 El circuito

Se empieza con un circuito nuevo, con un **Arduino Uno** y la **breadboard pequeña**. Las conexiones se hacen con criterio: en electrónica no existe **“al azar”**. Copien exactamente el dibujo.

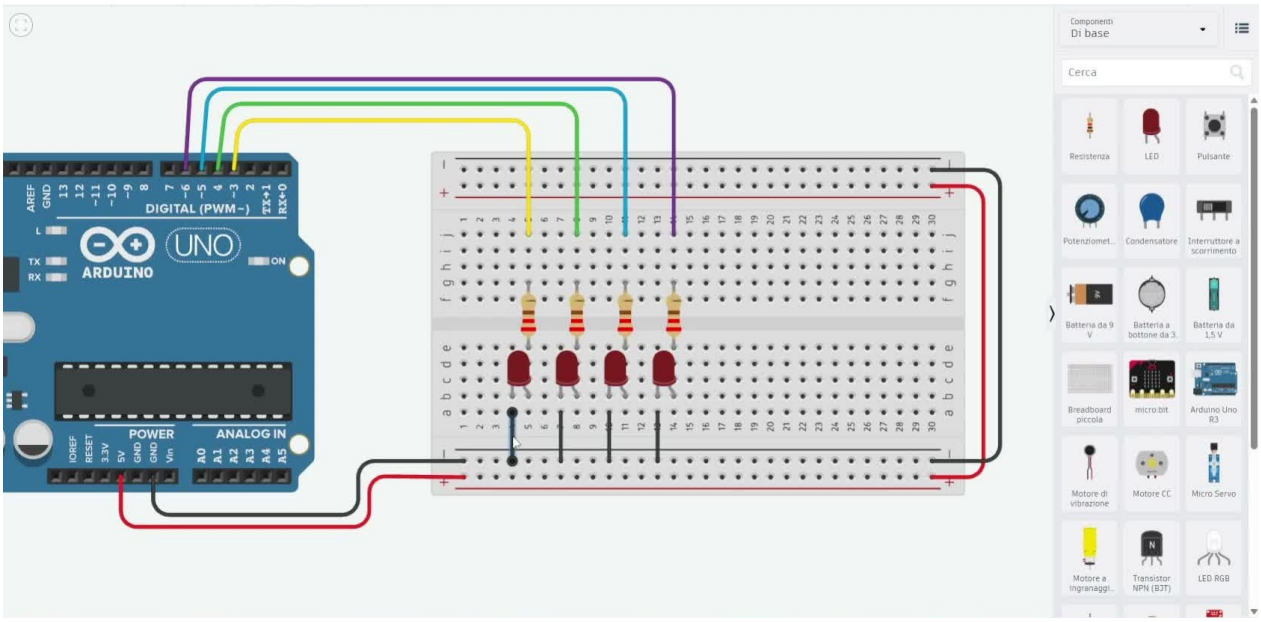


Figura 1: El circuito que hay que realizar: cuatro cables de colores desde los pines 3, 4, 5 y 6, cuatro resistores sobre el surco central, cuatro LED, los cables negros hacia el rail del negativo.

1. Cable **negro** desde **GND** de Arduino al rail − de abajo; cable **rojo** desde **5V** al rail + de abajo.
2. A la derecha de la breadboard, un cable rojo y uno negro conectan los rails de abajo con los **de arriba**. No los olviden.
3. Cables desde los **pines 3, 4, 5, 6** (amarillo, verde, celeste, violeta) a las **columnas 5, 8, 11, 14**, en la fila j de la parte de arriba: no en el rail rojo, sino en las columnas.
4. En las mismas columnas, cuatro **resistores** de **220 Ω** sobre el surco central, de la fila g a la fila e.
5. Debajo, cuatro **LED** en la fila b, con el **ánodo** (anodo) en la columna del resistor (5, 8, 11, 14) y el **cátodo** (catodo) en la columna de la izquierda (4, 7, 10, 13). Si se pone el ratón sobre una patita del LED, Tinkercad muestra la palabra **Anodo** o **Catodo**.
6. Desde el **cátodo** de cada LED, un cable **negro** desde la fila a hasta el rail −.

Atención

Atención a los **detalles**: los resistores son de **220 Ω**, no de 1 kΩ (y no 1220). Controlen los colores de las bandas: **rojo, rojo, marrón**. Escriban el valor y elijan la unidad Ω en el menú.

Nota

El color de los cables es un estándar. Los cables de **GND**, es decir el negativo, van siempre **negros**. El cable de **5V** va siempre **rojo**. No es que con otro color explote algo: es la **convención**. Cuando abren un circuito electrónico y ven un cable negro y uno rojo, saben que esas son las **alimentaciones**, es decir las tensiones estables. Para los demás cables elijan ustedes, **evitando negro y rojo**.

Nota

Cables torcidos. Si un cable tiene puntos en el medio, como una línea quebrada, seleccionen el punto y presionen **Canc** (Supr): el cable vuelve a estar recto. En Tinkercad también se selecciona y se borra un cable entero puesto por error.

3 El código, escrito desde cero

Se presiona **Codice** (Código) y de los **bloques** se pasa a **Testo** (Texto). Tinkercad propone un programa ya listo (`C++ code`, con `LED_BUILTIN` y `delay(1000)`): borramos todo y volvemos a escribir lo que necesitamos.

3.1 Los dos bloques fundamentales

Para funcionar, Arduino necesita **dos bloques de código**. Un **bloque** es algo que empieza con la palabra `void`, tiene un nombre (`setup` o `loop`), dos paréntesis redondos y las **llaves** (graffe). Todo lo que está dentro de las llaves forma parte de ese bloque.

Definición

`void setup()` se ejecuta **una sola vez**, al arrancar la máquina. Normalmente contiene las **configuraciones**.
`void loop()` se **repite infinitas veces**, como dice su nombre. Contiene las **operaciones** que hay que repetir.

Nota

Cómo se escriben las llaves en un teclado italiano: tecla **AltGr** + **Mayúscula** + **è** para la llave abierta `{`, tecla **AltGr** + **Mayúscula** + **+** para la llave cerrada `}`. Son las mismas teclas de los corchetes, que se hacen sin Mayúscula. La `S` de `setup` es minúscula.

3.2 Dentro de setup: pinMode

Entre las configuraciones hay algo que se hace siempre: decir si un pin (o terminal, o patita: es lo mismo) es una **entrada** o una **salida**. Es la máquina la que les dice a los LED si encenderse o apagarse, así que la dirección va **de la máquina hacia afuera**: son todas **salidas**. Para cada pin se usa `pinMode`, con el número del pin y la palabra `OUTPUT`:

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
}
```

La corriente **saldrá** del pin. Si hubiéramos conectado un **pulsador**, para saber que alguien lo presiona, habríamos escrito `INPUT`. Por ahora son todos `OUTPUT`.

Atención

Las **mayúsculas** cuentan: la **M** de `pinMode` y todo `OUTPUT`. Escritos en minúscula, la máquina no los reconoce.

3.3 Dentro de loop: encender un LED a la vez

Ahora queremos encender **un LED a la vez**: el 3, luego el 4, el 5, el 6, con una espera de **200 milisegundos** en cada paso. Hacen falta dos comandos:

- `digitalWrite(pin, valore)`: `HIGH` enciende, `LOW` apaga, **todo en mayúscula**;
- `delay(200)`: le dice a la máquina que se detenga 200 milésimas de segundo.

El razonamiento: enciendo el 3 y espero. Luego apago el 3, enciendo el 4 y espero. Luego apago el 4, enciendo el 5 y espero. Luego apago el 5, enciendo el 6 y espero. Al final apago el 6: el `loop` vuelve a empezar y enciende otra vez el 3.

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
}

void loop()
{
  // (*@\cmt{LOW apaga}@*)
  digitalWrite(3, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(4, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(5, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(6, LOW);
}
```

Una vez escrito el código, se presiona **Avvia simulazione** (iniciar simulación). Para modificarlo hay que presionar antes **Interrompi simulazione** (detener simulación).

Nota

Sin espera entre el apagado y el encendido siguiente. Si ponen un `delay` también después de cada `LOW`, el programa funciona, pero la luz se apaga, pasa un momento de oscuridad y solo después se enciende el LED siguiente. Tal como está escrito aquí, entre el apagado del 3 y el encendido del 4 no hay espera, y el movimiento es más **fluido**.

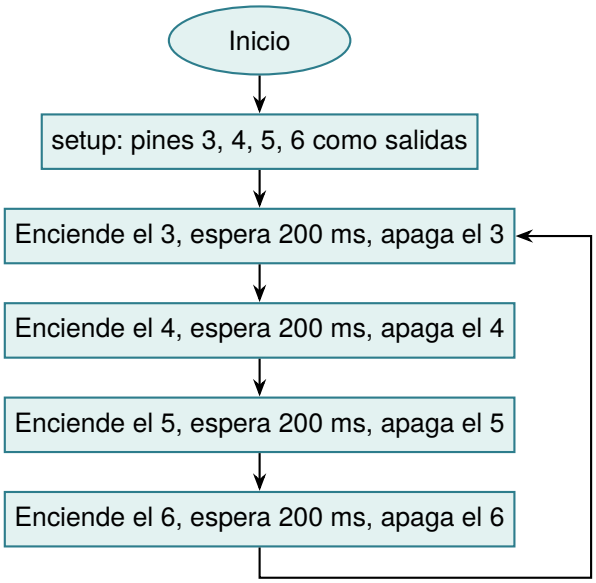


Figura 2: La secuencia: `loop` vuelve siempre a empezar desde el LED 3.

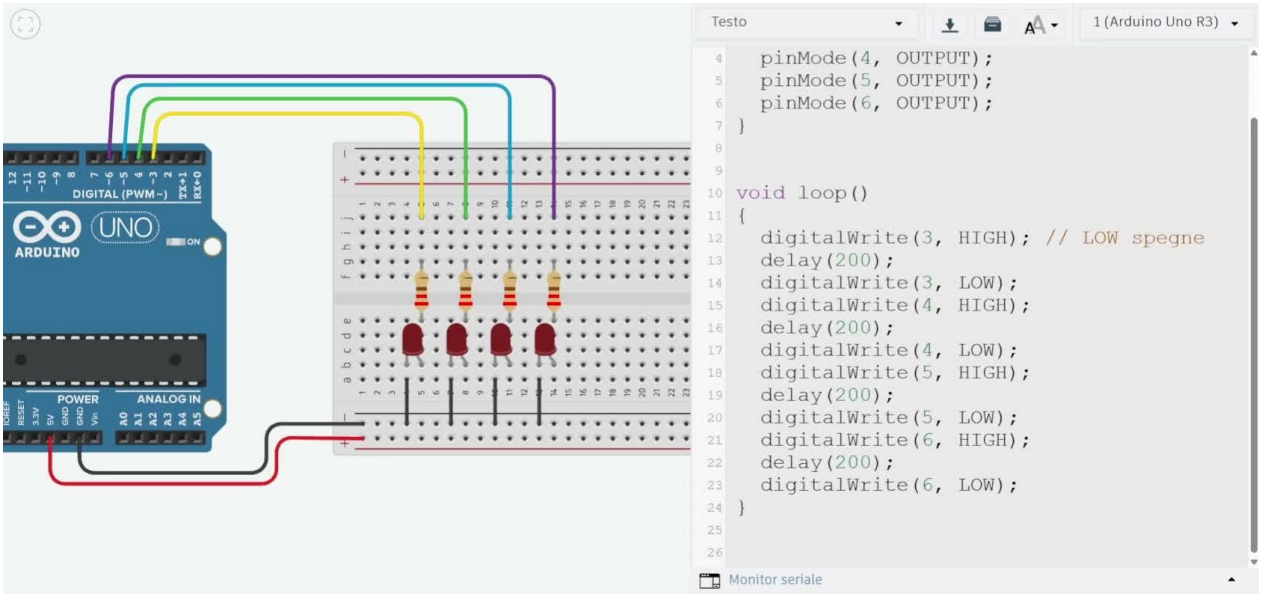


Figura 3: El resultado en Tinkercad, con el código al lado del circuito.

4 El paso siguiente: ocho LED, ida y vuelta

Quien ha terminado pasa a **8 LED**: se agregan otros cuatro **al lado**, en los **pines 7, 8, 9, 10**, cada uno con su resistor. La luz debe correr de izquierda a derecha y después **volver atrás**. El setup debe decir que también los pines 7, 8, 9 y 10 son salidas.

Atención

Los **cables rojos** están reservados a las alimentaciones: para los nuevos cables de señal usen otros colores, como el rosa, y eviten el negro y el rojo.

Para volver atrás no hace falta nada nuevo: se repiten las mismas operaciones **al revés**. Después del pin 10 se enciende el 9, luego el 8, hasta el 4. No hace falta volver a encender el 3 al final, porque lo hace el `loop` al empezar de nuevo.

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
}

void loop()
{
  // (*@\cmt{la luz va adelante, del 3 al 10}@*)
  digitalWrite(3, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(4, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(5, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(6, LOW);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(7, LOW);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);
  delay(200);
  digitalWrite(9, LOW);
  digitalWrite(10, HIGH);
  delay(200);
  digitalWrite(10, LOW);
  // (*@\cmt{la luz vuelve atr\'as, del 9 al 4}@*)
  digitalWrite(9, HIGH);
  delay(200);
  digitalWrite(9, LOW);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(8, LOW);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(7, LOW);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(6, LOW);
  digitalWrite(5, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(4, HIGH);
  delay(200);
  digitalWrite(4, LOW);
}
```

5 Cuando algo no funciona

El simulador **dice qué está mal**: miren **debajo del código**, donde en rojo aparece el error con el **número de línea**. Léanlo antes de preguntar.

- `expected ';' before ...`: **expected** quiere decir “me habría esperado” y **before** quiere decir “antes de”. Falta el **punto y coma** al final de la línea anterior, a menudo después de un

`delay`: en el programa de arriba, por ejemplo, en las líneas de los `delay`. Cada comando termina con `;`.

- `digitalWrite` mal escrito: “digital right” no existe. La **W** es mayúscula, el resto minúscula, y se escribe **Write**.
- `LOW` se escribe **todo en mayúscula**.
- Todo en rojo después de `loop`: faltan los **paréntesis redondos** después del nombre.
- Una **llave** de más o fuera de lugar: controlen la línea indicada.
- Dos `setup`: tiene que haber **uno solo**. No se duplica: se agregan las líneas en el que ya existe. Lo mismo vale para `loop`.
- El LED nuevo no se enciende: falta su `pinMode` en el setup, o los **cables negro y rojo** de los rails están mal conectados. Comparen el dibujo pieza por pieza.
- La luz va al revés: controlen qué pin conectaron a qué columna.
- La simulación parece detenida: miren el **Tempo simulatore** (tiempo del simulador) arriba; si avanza, está funcionando. A veces hacen falta unos segundos. Si la página no se actualiza, minimicen la ventana y vuelvan a abrirla.

Nota

El proyecto se **guarda solo**. Arriba se le puede dar el **nombre** que quieran; después se puede cerrar y volver a encontrarlo intacto.

6 La tarea para casa

La tarea es la **C02**, con el texto aparte: en el circuito de **8 LED**, dos luces que salen de los lados, van hacia el **centro**, “chocan” y **vuelven atrás**. La entrega es una **captura de pantalla** (screenshot) de la página en Classroom, **sin enlace**: para evaluar bastan el circuito y el código.

7 Qué recordar de esta lección

- Copiar el circuito con **criterio**: GND **negro**, 5V **rojo**, resistores de **220 Ω**.
- Un **bloque** es `void`, un nombre, los paréntesis redondos y las llaves.
- `setup` se ejecuta **una vez** y contiene las configuraciones; `loop` se **repite** al infinito.
- `pinMode(pin, OUTPUT)` para cada pin usado; `digitalWrite(pin, HIGH)` enciende, `LOW` apaga; `delay(ms)` espera.
- Para que la luz corra: apaga el LED actual y enciende el siguiente, **sin espera** en el medio, y después espera.
- Los errores se leen **debajo del código**: línea y mensaje. **expected** quiere decir “me esperaba”.

8 Glosario: palabras en italiano que se usan en clase

- **filo / fili**: cable / cables.
- **resistore**: resistor.
- **anodo, catodo**: ánodo, cátodo.
- **piedino**: patita, pin.
- **graffa / graffe**: llave / llaves.
- **tonde**: redondos (paréntesis).
- **uscita, ingresso**: salida, entrada.
- **accendere, spegnere**: encender, apagar.
- **punto e virgola**: punto y coma.
- **riga**: línea.
- **colonna**: columna.
- **solco**: surco.
- **avanti, indietro**: adelante, atrás.
- **compito**: tarea.
- **consegna**: entrega.